ERHVERVSAKADEMI AARHUS

Opgaver

Opgave 1

I denne opgave skal du anvende tabellen Konto (BankOpgave.sql på Canvas).

Du skal nu fra Java lave et program som kan overføre penge mellem to konti. Programmet **skal** laves efter følgende skabelon (Husk at du skal bruge samme connection-objekt gennem hele programmet og at det ikke må closes før til slut).

Indlæs frakonto fra bruger
Indlæs tilkonto fra bruger
Indlæs beløb fra bruger
Check at frakonto eksisterer og find saldoen
Kontroller at saldo er større end det beløb, der ønskes overført
Check at tilkonto eksisterer og find saldoen
Beregn de nye saldi
Opdater frakonto med den nye saldo
Opdater tilkonto med den nye saldo

Test at programmet virker – du må ikke gå videre, før programmet er testet.

Opgave 2

Nu skal du modificere programmet du lige har lavet.

Transaktioner kan startes fra Java på to måder. Den ene er via de metoder, der findes på Connection – i denne situation er det JDBC-driveren der sendes de faktiske kommandoer til databasen. Den anden måde er selv at sende kommandoerne direkte til SQL-server via execute kommandoen.

Herunder er vist begge muligheder (jeg antager, der et Connection-objekt *con* og et statement-objekt *stmt)*.

Start af en transaktion:

Via JDBC:	con.setAutoCommit(**false**)
Direkte til SQL-server	stmt.execute(”begin tran”)

Rollback af en transaktion:

Via JDBC:	con.rollback()
Direkte til SQL-server	stmt.execute(”rollback tran”)

Commit af en transaktion:

Via JDBC:	con.commit()
Direkte til SQL-server	stmt.execute(”commit tran”)

Bemærk at hvis du siger commit/rollback direkte til SQL-server skal der være startet en transaktion ellers får du en fejl – via JDBC kan du derimod godt sige commit/rollback en gang for meget – den sidste commit/rollback gør så bare ingenting.

Prøv at sætte start og afslutning af dine transaktioner ind i programmet og test det igen!

Test derefter programmet, hvor dit program er startet to gange. For helt at kunne styre afviklingen er det nødvendigt at lave en tastaturindlæsning mellem den sidste select og den første update.

Hvis resultatet er forkert, virker det som det skal. I opgave 3 reparerer vi problemet!

Opgave 3

Denne opgave er en fortsættelse af opgave 1 og 2.

I denne opgave skal du arbejde med Isolation levels.

Da du testede dit program i opgave 2 opdagede du, at programmet ikke forhindrede ukorrekte pengeoverførsler (samtidighedsproblemer). Dette skyldes, at der som default anvendes et meget lavt isolation level. Der kan anvendes følgende isolation levels

- READ UNCOMMITTED
- READ COMMITTED (default)
- REPEATABLE READ
- SERIALIZABLE

Man kan sætte isolation-levels på følgende måde (isolation-level sættes inden transaktionen starter)

Via JDBC:

```
con.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE)
```

Direkte til SQL-server:

```
stmt.execute(”set transaction isolation level serializable”)
```

Prøv at sætte dit isolation-level til henholdsvis REPEATABLE READ og SERIALIZABLE og undersøg om programmet nu giver det rigtige resultat.